Assignment 3

Q1. Docker Volume Persistence

Create a Docker volume named `data_vol`.

```
student@student-VMware-Virtual-Platform:~$ docker volume create data_vol
data_vol
```

Run a container using this volume mounted at `/data`.

```
student@student-VMware-Virtual-Platform:~$ docker run --name container_1 -it -v data_vol:/data busybox sh
/ # ls
bin    data   dev    etc    home   lib    lib64  proc   root   sys    tmp    usr    var
/ # ls data/
/ # cd  data/
/data # ls
/data # cat > main.txt
volume data successfully fetched
exit
/data # exit
```

Create a file inside `/data`, remove the container, and start a new container using the same volume.

```
student@student-VMware-Virtual-Platform:~$ docker stop container_1
container_1
student@student-VMware-Virtual-Platform:~$ docker rm container_1
container_1
```

Verify whether the file still exists.

```
student@student-VMware-Virtual-Platform:~$ docker run --name container_2 -it -v data_vol:/data busybox sh
/ # ls data/
main.txt
/ # cat data/main.txt
volume data successfully fetched
exit
```

Q2. Dockerfile – Python Application

Create a simple Python program that prints a message.

```
student@student-VMware-Virtual-Platform:~$ mkdir docker_image
student@student-VMware-Virtual-Platform:~$ cd docker_image/
student@student-VMware-Virtual-Platform:~/docker_image$ cat > message.py
print("Image created successfully bro....")
student@student-VMware-Virtual-Platform:~/docker_image$ nano Dockerfile
```

Write a Dockerfile to build an image and run the Python script inside a container.

```
  GNU nano 8.7.1
FROM python:3.9-slim-buster
WORKDIR /app
COPY message.py /app
CMD ["python","message.py"]
```

```
student@student-VMware-Virtual-Platform:~/docker_image$ docker build -t my-image .
[+] Building 3.1s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 128B
 => [internal] load metadata for docker.io/library/python:3.9-slim-buster
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/3] FROM docker.io/library/python:3.9-slim-buster@sha256:320a7a4250aba4249f458872adecf92eea88dc6abd
 => => resolve docker.io/library/python:3.9-slim-buster@sha256:320a7a4250aba4249f458872adecf92eea88dc6abd
 => [internal] load build context
 => => transferring context: 31B
 => CACHED [2/3] WORKDIR /app
 => [3/3] COPY message.py /app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:9130bfb14aff8f0fc049329acc3226a33b91aff13c3a98656345e8ac288cefbb
 => => exporting config sha256:f1ac448467597c17450da3d1e48f31d9a3c7bc4e88351d9195fa537f5c327159
 => => exporting attestation manifest sha256:e95f247bf675b6ea3ff1ee6438bd0ba15413da353bd27413739d27e6537c
 => => exporting manifest list sha256:078ca950015fe24fabfe92f217ddd91add10c248bfc23b3555262c57b9a721dd
 => => naming to docker.io/library/my-image:latest
 => => unpacking to docker.io/library/my-image:latest
student@student-VMware-Virtual-Platform:~/docker_image$ docker run my-image
Image created successfully bro....
```

Q3. Dockerfile – Java Application

Write a simple Java program that prints a message.

```
  GNU nano 8.7.1
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, bhai..ho gaya success");
    }
}
```

```
  GNU nano 8.7.1
FROM eclipse-temurin:26-jdk
WORKDIR /app
COPY message.java /app
CMD ["java","message.java"]
```

Create a Dockerfile to compile and run the Java program inside a container.

```
student@student-VMware-Virtual-Platform:~$ mkdir java_image
student@student-VMware-Virtual-Platform:~$ cd java_image/
student@student-VMware-Virtual-Platform:~/java_image$ nano message.java
student@student-VMware-Virtual-Platform:~/java_image$ nano Dockerfile
student@student-VMware-Virtual-Platform:~/java_image$ docker build -t my-image-java .
[+] Building 15.7s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 129B
 => [internal] load metadata for docker.io/library/eclipse-temurin:26-jdk
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/3] FROM docker.io/library/eclipse-temurin:26-jdk@sha256:a45ef8c9f6304700665c5a5ac7250122e951831593bf
 => => resolve docker.io/library/eclipse-temurin:26-jdk@sha256:a45ef8c9f6304700665c5a5ac7250122e951831593bf
 => => extracting sha256:edafa32f5d9e4d9629f0dbc6c01e63c79ff08133c01cfb7d3363aa8bd8a614cf
 => => extracting sha256:f3f25b84a7d74315d94ee4108231f8f5e0cd2a44725a7204d5dd6ce91c151697
 => => extracting sha256:4f7fcd351091da699a921b3ea35c3eed87873cc1315a31406299e27e1c421bb0
 => => extracting sha256:d3a111a9e59a5314b69db3ac62a6a4e8d34398b421583ff810b09833690ab30c
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1
 => => extracting sha256:edb43c31f817a55e9a105a70bfa17eb02673deaffe2eea5719da503afb084135
 => [internal] load build context
 => => transferring context: 180B
 => [2/3] WORKDIR /app
 => [3/3] COPY message.java /app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:3a2ba05217822ca48a0987dc3311a0712e8d0e665181a496b01082a63b419ddd
 => => exporting config sha256:787a70157145f8ff5b82b5a042129baab772966b5ce1e5b0d6e92ac47ca19650
 => => exporting attestation manifest sha256:90bff8bf115c592a15b31b95509f8ac6c924758b7f6b78a0d4b1dba20acfb9
 => => exporting manifest list sha256:49c78921ef1199d65687de25b10ae31f134914b0e94f49ff05cc309b4eb162aa
 => => naming to docker.io/library/my-image-java:latest
 => => unpacking to docker.io/library/my-image-java:latest
student@student-VMware-Virtual-Platform:~/java_image$ docker run my-image-java
Hello, bhai..ho gaya success
```

Q4. Docker Volume with MySQL

Run a MySQL container using a Docker volume for data storage.

```
student@student-VMware-Virtual-Platform:~$ docker run -d -e MYSQL_ROOT_PASSWORD=root123 -v mysql_vol:/var/lib/mysql -p 3300:3306 mysql
8a2494aeaf40a8673153a9ba2dd3f3ebdf2d655545f5d3242f489a53991d68e0
student@student-VMware-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS
8a2494aeaf40   mysql     "docker-entrypoint.s…"   About a minute ago   Up About a minute   33060/tcp, 0.0.0.0:3300->3306/tcp, [::]:3300->3
student@student-VMware-Virtual-Platform:~$ docker exec -it stoic_babbage mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 26.10.0 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database docker_ass;
Query OK, 1 row affected (0.005 sec)
```

```
mysql> use docker_ass;
Database changed
mysql> create table image (port varchar(10),image varchar(10));
Query OK, 0 rows affected (0.011 sec)

mysql> insert into image values ("3306","MySQL");
Query OK, 1 row affected (0.021 sec)

mysql> select * from image;
+------+-------+
| port | image |
+------+-------+
| 3306 | MySQL |
+------+-------+
1 row in set (0.000 sec)

mysql> exit
Bye
```

Create a database and table, insert some records, then restart the container.

Verify whether the data persists after restart.

```
student@student-VMware-Virtual-Platform:~$ docker stop stoic_babbage
stoic_babbage
student@student-VMware-Virtual-Platform:~$ docker restart stoic_babbage
stoic_babbage
student@student-VMware-Virtual-Platform:~$ docker exec -it stoic_babbage mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| docker_ass         |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.009 sec)

mysql> use docker_ass;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from image;
+------+-------+
| port | image |
+------+-------+
| 3306 | MySQL |
+------+-------+
1 row in set (0.000 sec)
```

Q5. Dockerfile – Nginx Web Server

Create a custom HTML page.

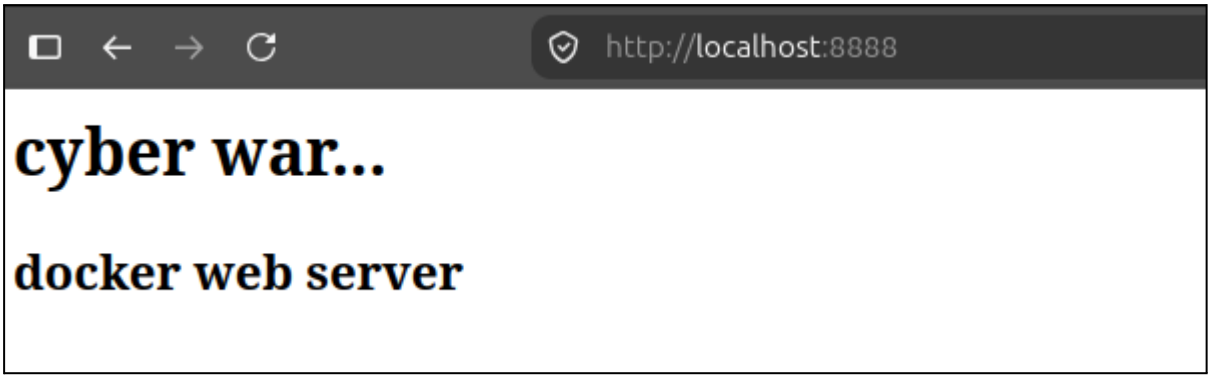
Write a Dockerfile to serve this page using an Nginx container.

```
  GNU nano 8.7.1
FROM nginx
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

```
student@student-VMware-Virtual-Platform:~/my-web$ docker build -t my-web .
[+] Building 0.6s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 107B
 => [internal] load metadata for docker.io/library/nginx:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 31B
 => CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8ee
 => => resolve docker.io/library/nginx:latest@sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce50
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:e8c4062254b417519cfb7fc725a86f130ed68bb1986985a2bb252a67606ee24e
 => => exporting config sha256:ef0ac8d2342f0ed5eacda1640ba1b1133905b21aca675edb8822bcd418ab2b0c
 => => exporting attestation manifest sha256:dc59dd7f5b695c7942294bcbe4b6aa21d4df658b67754bbc945a3191ee2328a3
 => => exporting manifest list sha256:a3ddad073757e849eb0443762daac6966f38184d020f7c6f8a82ec81904c4e77
 => => naming to docker.io/library/my-web:latest
 => => unpacking to docker.io/library/my-web:latest
```

```
student@student-VMware-Virtual-Platform:~/my-web$ docker run -d --name my-web -p 8888:80 my-web
730cab4c12f6a7c5c812bd0f41610e7da80a05dcd8640dc4116be96c0c155539
```

Run the container and verify the output in a browser.



Q6. Bind Mount vs Volume

Create a directory on the host system and store a file in it.

Run a container using a bind mount to access this directory.

Compare this with using a Docker volume and explain the difference.

```
student@student-VMware-Virtual-Platform:~$ mkdir bind_mount
student@student-VMware-Virtual-Platform:~$ cd bind_mount/
student@student-VMware-Virtual-Platform:~/bind_mount$ cat > file_secret.txt
secret bicret kuch nahi he bro...
student@student-VMware-Virtual-Platform:~/bind_mount$ docker run -it  --name bind_image -v ${PWD}:/data busybox
/ # ls
bin    data   dev    etc    home   lib    lib64  proc   root   sys    tmp    usr    var
/ # cat data/file_secret.txt
secret bicret kuch nahi he bro...
/ # exit
```

Q7. Dockerfile – Shell Script Execution

Write a shell script that prints a message and system date.

```
  GNU nano 8.7.1
S#!/bin/bash

echo "Hello bro...."
echo "You know the todays date -> "
echo "i know you dont know so ye le -> $(date)"
```

Create a Dockerfile to execute this script inside a container.

```
  GNU nano 8.7.1
FROM ubuntu:latest
WORKDIR /my-script
COPY bash1.sh .
RUN chmod +x bash1.sh
CMD ["/bin/bash","bash1.sh"]
```

Q8. Shared Volume Between Containers

Create a Docker volume and use it with two different containers.

```
student@student-VMware-Virtual-Platform:~/shared_cont$ docker volume create shared_vol
shared_vol
student@student-VMware-Virtual-Platform:~/shared_cont$ docker run -it -v shared_vol:/data
 busybox
/ # cd data/
/data # ls
/data # cat > new.txt
hii broo..
/data # cat container2.txt
ye mear file he re tereko mila kya...
/data #
```

Write data from one container and access the same data from another container.

Verify that both containers can share the same data.

